

Ejercicio 1: Juego de cartas (10 ptos)

ESTE EJERCICIO EVALÚA LA RA2 DEL MÓDULO EN SU TOTALIDAD
ESTE EJERCICIO EVALÚA LA RA6 DEL MÓDULO EN SU TOTALIDAD
ESTE EJERCICIO RECUPERA LA RA1 DEL MÓDULO EN SU TOTALIDAD
ESTE EJERCICIO RECUPERA LA RA3 DEL MÓDULO EN UN 50%

Objetivo: implementar un juego de cartas simple utilizando POO en Java.

El modelo de datos será el siguiente:

- clase Carta, que se caracterizará por el palo (“Oros”, “Copas”,...) y el número de carta
- clase Baraja, que se caracterizará por el conjunto (debe ser un Set) de cartas que contiene.
Dentro del constructor sin parámetros de la clase Baraja, se inicializará la baraja con cuarenta cartas, del 1 al 10 de cada palo **(1,5 pt.)**
- clase Jugador con los atributos nombre, puntuación y lista de cartas que tiene en la mano (debe ser un List)

Implementar los siguientes métodos en Jugador:

- void recibirCarta(Carta carta): añade la carta a su conjunto de cartas y la elimina desde la Baraja **(1 pt.)**
- Carta jugarCarta(String palo): selecciona de su mano la carta más alta de ese palo. La elimina de la lista y muestra el mensaje “Jugando el numero *número_de_carta* de *nombre_palo*”. Si no hay cartas de ese palo, no elimina nada y muestra el mensaje “Paso”. Además devuelve la carta jugada (y eliminada), null en el caso de no haber carta **(1,5 pt.)**
- void mostrarMano(): muestra por consola las cartas del jugador **(1 pt.)**

Crear una clase principal Juego que gestione la partida entre dos jugadores. Realizará las siguientes acciones.

1. Se crea un objeto Baraja nuevo.
2. Se crean dos objetos Jugador, con cualquier nombre y puntuación cero.
3. Cada jugador recibe 10 cartas al principio, escogidas aleatoriamente de la baraja **(1,5 pt.)**
4. El sistema elige un palo aleatoriamente.
5. Empieza el juego en el que de forma repetida se realizarán las siguientes acciones:
 - a. Cada jugador trata de jugar la carta más alta de ese palo:
 - Si los dos jugadores han podido jugar carta, el que ha jugado la carta más alta sube un punto su puntuación
 - Si un jugador no ha podido jugar carta y el otro sí, el que ha jugado carta sube un punto su puntuación
 - Si ninguno de los jugadores ha podido jugar carta, cada jugador recibe las cartas necesarias para que vuelva a tener 10 en la mano

El juego finaliza en el momento en el que un jugador coge la última carta de la baraja, es decir, esta se queda vacía. No habrá más jugadas después de ese momento, aún teniendo cartas en mano.

Condición de finalización del juego: **1,5 pt.** Funcionamiento de cada jugada: **2 pt.**

Ejercicio 2: Sistema de Criaturas Místicas (10 ptos)

ESTE EJERCICIO EVALÚA LA RA4 DEL MÓDULO EN SU TOTALIDAD
ESTE EJERCICIO EVALÚA LA RA7 DEL MÓDULO EN SU TOTALIDAD
ESTE EJERCICIO RECUPERA LA RA3 DEL MÓDULO EN UN 50%

En un mundo mágico, existen diferentes criaturas místicas con habilidades únicas. Debes diseñar un sistema que represente a estas criaturas utilizando herencia e implementación de interfaces.

Define una interfaz llamada PoderMístico con un método activarPoder(). **(1 pt)**

Crea una clase abstracta CriaturaMística que implemente la interfaz PoderMístico. **(1 pt)**

La clase abstracta debe tener atributos nombre y edad, y un método abstracto describirCriatura() que imprima la información básica de la criatura.

Declara las clases Dragon, Unicornio y Elfo, que hereden de criatura mística y añada cada una una propiedad característica de la criatura (ejemplo: para Unicornio, longitud del cuerno) **(2 pt)**

Cada clase debe tener su propia implementación del método describirCriatura() que incluya toda la información de la criatura.

Implementa el método activarPoder() de la interfaz PoderMístico de manera única para cada clase, mostrando un mensaje relacionado con sus poderes.

En el programa principal, crea un array de cuatro criaturas místicas que incluya al menos una instancia de cada clase concreta. **(2 pt)**

Posteriormente mostrar la descripción de todas las criaturas **(2 pt)** y activar el poder solo de los Elfos. **(2 pt)**